INTERNSHIP PROJECT REPORT

Brain Tumor MRI Image Classification

Custom CNN versus transfer learning with Grad-CAM explainability and a Streamlit application

Prepared by	P Suman Sangeet (PGDM - Big Data Analytics)
Programme	INNOVEXIS Data Science & Gen AI Internship - Project 9
Domain	Medical imaging - deep learning image classification
Date	October 2026
Live application	Streamlit app
Repository	github.com/SUMANSANGEET/Brain-Tumor-MRI-Classification

Research prototype - not a medical device. All results come from a single public dataset and must not be used for clinical decisions. Figures in this document are taken from the project notebook's executed outputs.

Abstract

This report documents Project 9 of the INNOVEXIS Data Science & Gen AI Internship: an end-to-end deep-learning system that classifies brain MRI slices into four categories - glioma, meningioma, pituitary tumor and no tumor. Using 2,443 labelled images, the project built a custom convolutional neural network from scratch and compared it with three ImageNet transfer-learning backbones (MobileNetV2, EfficientNetB0, ResNet50V2), each at baseline and tuned stages. A data audit revealed that the vendor's train/valid/test split leaked augmented copies of the same scan across folders, so all images were pooled and re-split by source-scan ID. The final model, a fine-tuned ResNet50V2, reaches 87.5% accuracy, 0.864 macro-F1 and 0.976 one-vs-rest ROC-AUC on 407 untouched test images, with meningioma the weakest class (recall 64.8%). The model is deployed in a Streamlit application offering confidence analytics, Grad-CAM explanations and a stability check. The work is a research prototype and not a medical device.

Contents

1	Introduction and objectives	8	Deployment
2	Internship context and deliverables	9	Challenges and how they were solved
3	Dataset	10	Learning outcomes
4	Exploratory and statistical analysis	11	Limitations and ethics
5	Methodology	12	Future work
6	Results	13	Conclusion
7	Explainability	A	Appendices

1. Introduction

Brain tumors are diagnosed largely from MRI, and radiologists face growing worklists in which distinguishing tumor types from normal scans is time-consuming and subject to inter-reader variability. Deep learning has shown promise for this task, but medical-imaging models are easy to over-state: small datasets, near-duplicate images and acquisition artefacts can all inflate accuracy. This project therefore treats *honest evaluation* as a core objective alongside predictive performance.

1.1 Problem statement

Build a deep-learning system that (1) classifies a brain MRI into glioma, meningioma, pituitary or no tumor; (2) compares a custom CNN built from scratch with transfer-learning backbones; (3) is evaluated with a leakage-free split, macro-averaged metrics, confusion matrices and Grad-CAM; and (4) is deployed as a Streamlit app returning the predicted class with a confidence score.

1.2 Objectives

- Understand the dataset: class balance, image resolution, quality and hidden risks.
- Preprocess and augment images for CNN training.
- Design and train a custom CNN with batch normalisation and dropout.
- Fine-tune pretrained backbones and compare all models on identical test data.
- Select a deployment model using validation data only, with efficiency as a tie-breaker.
- Explain predictions with Grad-CAM and deploy a user-friendly web application.

2. Internship context and deliverables

The project was completed as an individual contribution within the INNOVEXIS Data Science & Gen AI Internship (Project 9) by P Suman Sangeet, PGDM student in Big Data Analytics. The brief specified a 14-day timeline and these deliverables: trained custom-CNN and pretrained models (.h5), a Streamlit application, notebooks/scripts for training, evaluation and deployment, a model comparison, and a public GitHub repository with README and clean, modular code. All were produced; the repository also adds a src/ package (config, dataset setup, audit, training, evaluation utilities) and a deployed live demo.

3. Dataset

The data is the Roboflow-hosted *Labeled MRI Brain Tumor Dataset* (CC BY 4.0), containing 2,443 MRI images in four classes. Labels follow a standardised protocol according to the dataset documentation. Every image is a 640 x 640 JPEG.

Class	Vendor train	Vendor valid	Vendor test	Total	Share
Glioma	564	161	80	805	32.9%
Pituitary	438	118	54	610	25.0%
Meningioma	358	124	63	545	22.3%
No tumor	335	99	49	483	19.8%
Total	**1,695**	**502**	**246**	**2,443**	100%

3.1 Data quality checks

- No missing metadata, unreadable files or byte-identical duplicates (MD5 hashing).
- Uniform dimensions (640 x 640) and effectively grayscale content stored as RGB.

- 352 source scans appear twice and 164 source IDs span multiple splits - a leakage risk found by parsing the Roboflow source ID from file names.

4. Exploratory and statistical analysis

The notebook contains 15 interactive charts organised by the Univariate-Bivariate-Multivariate (UBM) rule, each answering why the chart was chosen, what insight it gives and its business impact. Per-image descriptors (brightness, contrast, foreground ratio, sharpness via Laplacian variance, entropy, chroma) were computed from a 128 x 128 grayscale view.

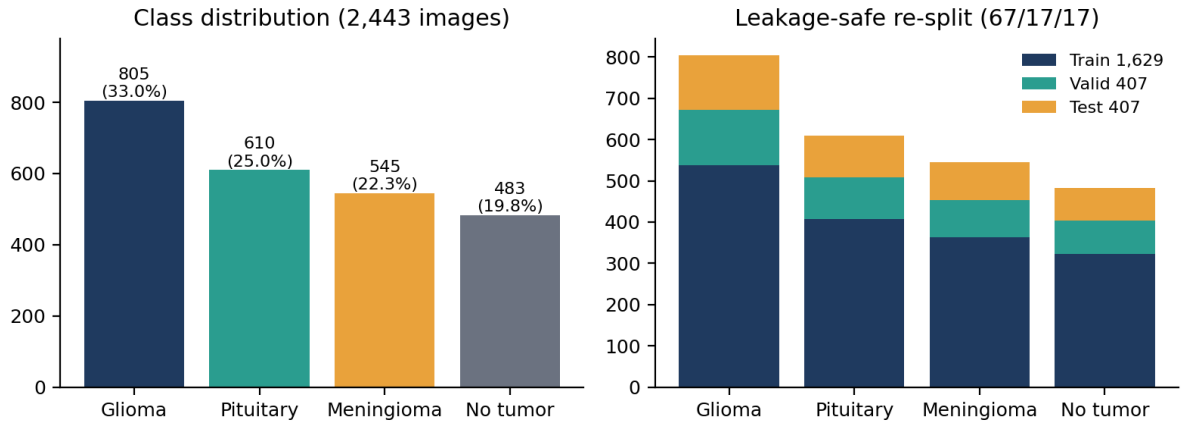


Figure 1 - Class distribution and the leakage-safe split.

4.1 Key EDA findings

- Mean brightness differs strongly by class (glioma ~28, meningioma ~46, pituitary ~50, no tumor ~60), as does foreground coverage (0.41 / 0.51 / 0.66 / 0.61).
- Class-level differences in brightness, contrast, sharpness and file size are statistically overwhelming ($p < 1e-100$).
- A metadata-only Random Forest achieves 80.7% accuracy (0.800 macro-F1) with grouped 5-fold CV - evidence of a shortcut a CNN could exploit.

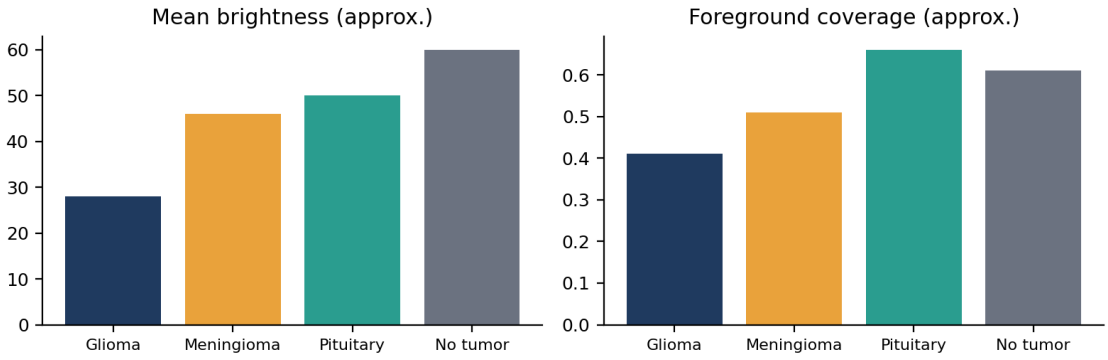


Figure 2 - Class means of brightness and foreground coverage.

4.2 Hypothesis tests

Test	Question	Result
Kruskal-Wallis + Bonferroni post-hoc	Does brightness differ across the four classes?	Reject H0 - large effect; skewed distributions justified the rank-based test
One-way ANOVA + Tukey HSD (Levene, eta-squared)	Does contrast differ across classes?	Reject H0 - variances unequal, so effect size read alongside H1

Test	Question	Result
Chi-square (contingency table)	Are vendor split proportions consistent across classes?	p ~ 0.47 - fail to reject; splits are stratified, but this says nothing about leakage

5. Methodology

5.1 Leakage-safe splitting

All 2,443 images were pooled and split with StratifiedGroupKFold using the source-scan ID as the group, giving 1,629 training, 407 validation and 407 test images (about 67 / 17 / 17) with identical class stratification. The code asserts zero source overlap between splits. The test set is touched only once for final reporting; validation data drives early stopping, tuning and model selection.

5.2 Preprocessing, augmentation and imbalance

Images are resized to 224 x 224 and scaled per backbone ([-1, 1] for MobileNetV2, raw 0-255 for EfficientNetB0 which normalises internally, and the matching preprocessing for ResNet50V2). Augmentation (flips, rotation, zoom, translation, brightness, contrast) lives inside the model so it is active only during training. Class imbalance is handled with inverse-frequency class weights.

5.3 Model architectures

Model	Description	Parameters
Custom CNN	Four convolutional stages (32-64-128-256 filters), Conv3x3-BatchNorm-ReLU blocks, max-pooling, global average pooling, dropout and dense head	0.65 M (tuned) / 1.16 M (baseline)
MobileNetV2	Depth-wise separable convolutions; frozen backbone with GAP-Dropout-Dense(256)-Dropout-Dense(4) head, then fine-tuning	2.6 - 2.9 M
EfficientNetB0	Compound-scaled MBConv blocks with squeeze-and-excitation; same head and recipe as MobileNetV2	4.4 - 4.7 M
ResNet50V2	Residual network with pre-activation blocks; largest capacity of the set	24.1 M

5.4 Training, tuning and selection

Training used EarlyStopping, ModelCheckpoint and ReduceLROnPlateau callbacks. Hyper-parameters were found with random search, validated by stratified-group K-fold cross-validation: the custom CNN end-to-end (best: 24 base filters, dense 64, dropout 0.5, learning rate ~0.003, L2 1e-5) and the transfer heads on cached embeddings (MobileNetV2 best head: 512 units, dropout 0.3, CV macro-F1 0.907 +/- 0.015; EfficientNetB0: 512 units, dropout 0.2, CV macro-F1 0.910 +/- 0.008). Top backbone layers were then fine-tuned at a low learning rate in two phases. The deployed model is the one with the highest **validation** macro-F1; if candidates fall within 0.5 percentage points, the lightest wins. All training ran on CPU (TensorFlow 2.21, Keras 3.15, Python 3.11).

6. Results

All numbers below are on the same 407-image, leakage-free test set.

Model	Stage	Acc.	Macro-F1	ROC-AUC	Val F1	Params (M)	ms / img
ResNet50V2	baseline	0.8747	0.8640	0.9760	0.8883	24.09	112.0
MobileNetV2	tuned	0.8624	0.8546	0.9729	0.8809	2.92	50.2
EfficientNetB0	tuned	0.8354	0.8232	0.9691	0.8603	4.71	67.2
MobileNetV2	baseline	0.8231	0.8165	0.9657	0.8518	2.59	57.3
EfficientNetB0	baseline	0.8157	0.7917	0.9687	0.8461	4.38	65.5
Custom CNN	tuned	0.6732	0.6470	0.9374	0.6607	0.65	25.4
Custom CNN	baseline	0.3784	0.2080	0.7839	0.2067	1.16	46.6

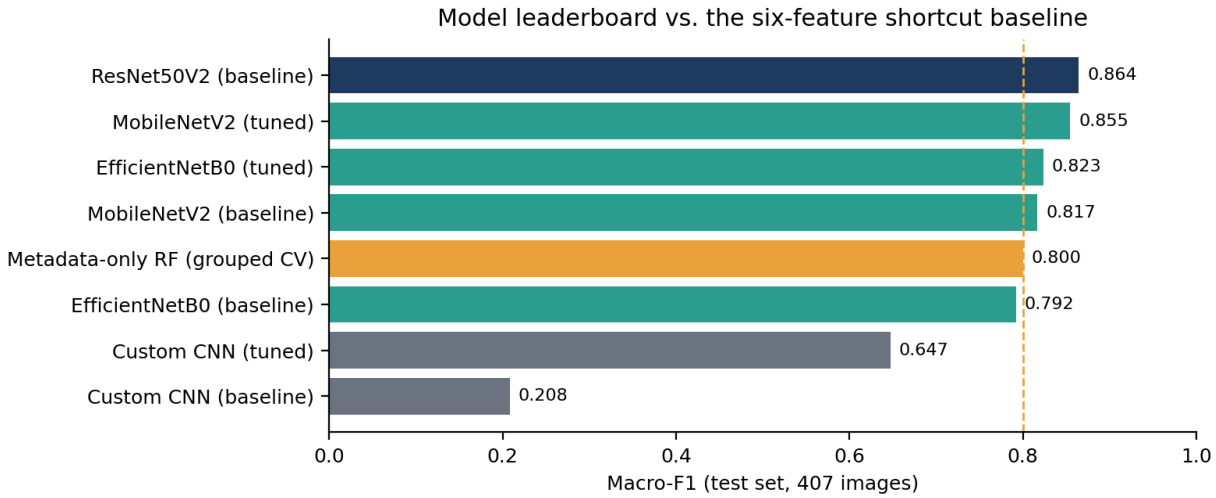


Figure 3 - Macro-F1 leaderboard with the metadata-only baseline.

6.1 Final model: per-class performance

Class	Precision	Recall	F1	Support
Glioma	0.932	0.918	0.925	134
Meningioma	0.868	0.648	0.742	91
No tumor	0.889	0.900	0.894	80
Pituitary	0.810	1.000	0.895	102
Macro average	**0.874**	**0.867**	**0.864**	407

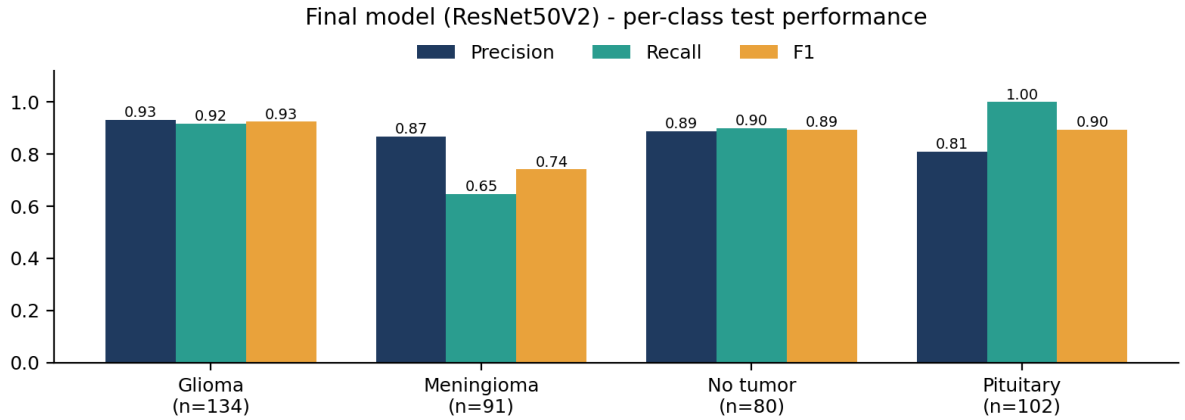


Figure 4 - Per-class precision, recall and F1 for ResNet50V2.

6.2 Discussion

- **Transfer learning wins.** With ~1,600 training images, pretrained backbones converged faster and generalised far better than the from-scratch CNN (0.864 vs 0.647 macro-F1 at best).
- **Tuning pays off for lightweight models:** +3.8 pp macro-F1 for MobileNetV2 and +3.2 pp for EfficientNetB0.
- **Efficiency trade-off:** tuned MobileNetV2 is 0.9 pp behind ResNet50V2 with 8.3x fewer parameters and 2.2x lower latency.
- **Meningioma** is the hardest class for every backbone (recall ~45-65%).
- **Shortcut check:** deep models beat the metadata-only baseline by about 6 macro-F1 points, which makes explainability review necessary.

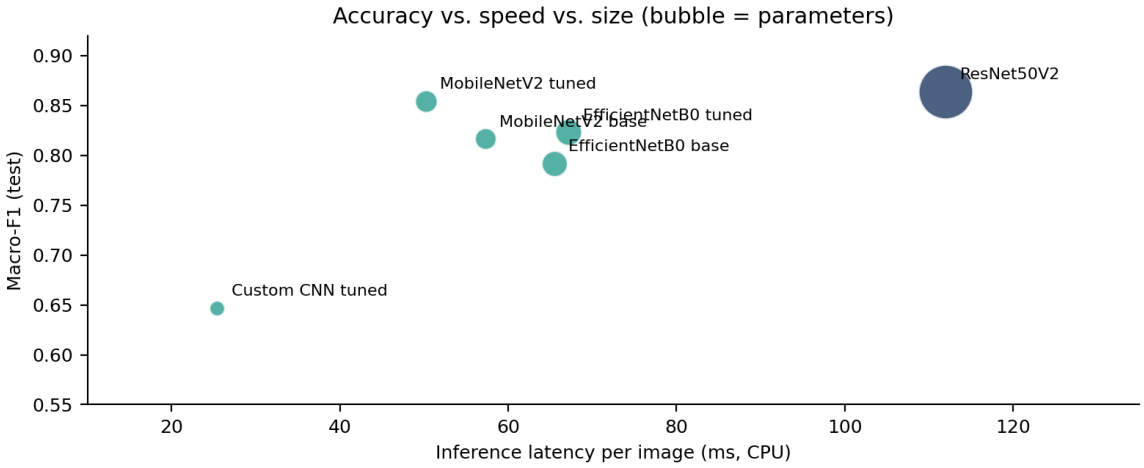


Figure 5 - Accuracy versus latency and size.

7. Explainability

Grad-CAM back-propagates the predicted-class score to the final convolutional feature maps, weights each map by its average gradient and overlays the resulting heatmap on the MRI. The notebook generates overlays for correct predictions and for the most confident mistakes. Healthy behaviour is attention on brain tissue or the lesion (for pituitary tumors, the sellar region); warning signs are attention on the black background, skull edge or image borders, which would indicate reliance on framing or acquisition artefacts - the risk quantified in Section 4. The Streamlit app exposes Grad-CAM for any class. Grad-CAM is an explanation technique, not proof of clinical correctness.

8. Deployment

The application (app.py, Streamlit 1.50) offers three tabs:

Tab	Capabilities
Live Diagnosis	Multi-file upload, webcam or sample gallery; predicted class, confidence gauge, class probabilities, margin, uncertainty, latency; test-time-augmentation stability check; triage message; Grad-CAM for any class; session log with CSV export
Model Performance	Leaderboard by macro-F1, accuracy-versus-latency trade-off, per-class precision/recall/F1, confusion matrix, training curves
About and Limitations	Pipeline, dataset balance and honest limitations; persistent 'not a medical device' warning in the sidebar

- **Model artefacts:** best_model.h5 (101 MB) plus model_meta.json (image size 224, class order, last convolutional layer, metrics). The .h5 is downloaded at start-up from a Hugging Face Hub repository via hf_hub_download.
- **Compatibility:** convert_model.py re-saves the model in native .keras format and verifies it; app.py uses a tolerant loader for version differences. Deployment dependencies are pinned in requirements.txt (tensorflow-cpu 2.16.2, numpy 1.26.4, scikit-learn 1.5.2, pandas 2.2.3) with Python 3.11.
- **Sanity check:** the saved model was reloaded and re-predicted on test images with 100% agreement and zero probability difference.

Live application: https://brain-tumor-mri-classification-abgwcmcbjt4dgfbbnbhh5y.streamlit.app/

9. Challenges and how they were solved

Challenge	Resolution
Hidden train/test leakage from augmented copies sharing a source scan	Parsed source IDs from file names, pooled all images and re-split with StratifiedGroupKFold; asserted zero overlap
Custom CNN unstable and slow to learn (baseline stuck at 37.8% accuracy; CV fold macro-F1 from 0.313 to 0.671)	Random-search tuning with BatchNorm, dropout, GAP and ReduceLROnPlateau lifted it to 67.3%; documented as a baseline rather than forced into the deployment role
CPU-only training (no GPU on native Windows); custom CNN tuning took about 407 minutes	Cached backbone embeddings for head search; efficient two-phase fine-tuning
Large model file (101 MB) and Keras/TensorFlow version differences between training and hosting	Hosted weights on Hugging Face Hub; native .keras conversion script; tolerant loader and pinned dependencies
Possible reliance on brightness / framing shortcuts	Metadata-only baseline for comparison; Grad-CAM review; group-aware cross-validation

10. Learning outcomes

- Designing leakage-aware validation for medical imaging.
- Building CNNs and applying transfer learning, fine-tuning and cross-validated hyper-parameter search with TensorFlow/Keras.
- Evaluating with macro-F1, per-class recall, ROC-AUC, confusion matrices and latency/size trade-offs.
- Applying statistical tests (Kruskal-Wallis, ANOVA, chi-square) to image-derived features.
- Explaining deep models with Grad-CAM and communicating uncertainty and limits.
- Packaging, hosting and deploying a model as an interactive Streamlit application with Git/GitHub.

11. Limitations and ethics

- Single public dataset of 2-D slices with no patient identifiers; patient-level splitting and multi-centre external validation are not possible with this data.
- Test-set noise: 80-134 images per class means one error moves class recall by roughly 1 pp.
- ResNet50V2 was evaluated at baseline stage only; tuned alternatives were not exhaustively matched.
- Predictions are probabilistic and uncalibrated; Grad-CAM does not establish causality.
- This is an educational/research prototype. A qualified medical professional must interpret MRI findings and make diagnostic decisions.

12. Future work

- Tune ResNet50V2 and compare it directly with tuned MobileNetV2 (a lighter deployment option).
- Probability calibration (temperature scaling) with an 'uncertain - refer to specialist' band.
- Test-time augmentation and ensembling; targeted meningioma improvements.
- Tumor segmentation or localisation (e.g. U-Net) and 3-D models.
- Bayesian hyper-parameter search, MLflow experiment tracking, Docker and CI/CD.

13. Conclusion

The project delivers a complete, reproducible pipeline - data audit, leakage-safe split, custom CNN versus three transfer-learning backbones, cross-validated tuning, explainability and a deployed application. Its central lesson is methodological: the vendor split would have overstated performance, and simple intensity statistics already score about 80% macro-F1, so the honest ResNet50V2 result of 0.864 macro-F1 is a modest but real improvement that still needs external validation. The system is best positioned as a triage and second-reader prototype, not a diagnostic tool.

Appendix A - Repository structure

Path	Purpose
app.py, .streamlit/	Streamlit application and configuration
src/	config.py, setup_dataset.py, audit_dataset.py, train.py, evaluate.py, utils.py
Brain Tumour MRI Classification.ipynb	Full analysis notebook (330 cells): EDA, tests, models, evaluation, Grad-CAM, export
artifacts/	leaderboard.csv, model_meta.json/joblib (large weights hosted externally)
reports/	dataset_audit.csv and earlier-run figures (to be refreshed)
Tumour/	Dataset: train / valid / test with class sub-folders

Path	Purpose
convert_model.py, requirements.txt, runtime.txt	Model conversion and deployment environment

Appendix B - Technology stack

Python 3.11; TensorFlow/Keras; NumPy, Pandas, scikit-learn; Matplotlib, Seaborn, Plotly; Pillow, OpenCV; Streamlit; Hugging Face Hub; Git/GitHub; Jupyter Notebook.

GitHub: https://github.com/SUMANSANGEET/Brain-Tumor-MRI-Classification